

Estrategias de usos de bucles

David Álvarez

4 de marzo de 2026

Índice

1. Introducción	1
2. Contadores	2
3. Acumuladores	3
4. Banderas (Flags)	6

1. Introducción

En el desarrollo de programas es habitual encontrarse ante la necesidad de contabilizar o detectar características en un conjunto de datos. También es común desarrollar programas que resuelvan ecuaciones de tipo sumatorio o productorio. Para resolver este tipo de situaciones existen estrategias que ayudan al proceso de desarrollo de código que calcula la solución buscada. Las más comunes son: contadores, acumuladores y banderas.

2. Contadores

Resulta habitual que entre los requisitos de un programa sea necesario **contabilizar el número de veces que se cumple una condición**. Por ejemplo, suponga que queremos determinar la cantidad de divisores que tiene un número entero. En estos casos se utilizan los contadores.

Un contador es una variable entera cuyo valor inicial es (normalmente) cero y que se incrementa en 1 cada vez que, durante un proceso iterativo (bucle), se cumple la condición que se está buscando (por ejemplo, que un número es divisor de otro).

Como se observa en el programa del ejemplo, la variable *divisores* desempeña el papel de contador. Durante la declaración se inicializa a cero y su valor se incrementa cuando, en cada iteración del bucle que recorre todos los enteros entre 1 y *n*, se comprueba que uno de los valores cumple la condición de divisibilidad.

```
/* Ejemplo: Contar los divisores de un número entero
   introducido por el usuario */

#include <stdio.h>

int main(void)
{
    int n, i, divisores=0;
    printf ("Introduzca un numero: ");
    scanf ("%d", &n);
    printf ("Divisores: ");

    for (i=1; i<=n; i++)
    {
        if (n % i == 0)
        {
            printf ("%d ",i);
            divisores++;
        }
    }
    printf ("\nEl numero %d tiene %d divisores\n", n, divisores);

    return 0;
}
```

3. Acumuladores

Los acumuladores permiten realizar operaciones de agregación. En ocasiones se utilizan para sumar aquellos datos que cumplen una condición. En otras representan un sumatorio o un productorio de series.

Los acumuladores son variables numéricas y su valor inicial es el elemento neutro de la operación de agregación, es decir, el 0 en el caso de sumas, y el 1 en el caso de productos.

```
/* Ejemplo: Suma de los números pares entre 1 y n */

#include <stdio.h>

int main(void)
{
    int n,i,suma=0;
    printf ("Introduzca un número: ");
    scanf ("%d",&n);
    for (i=1;i<=n;i++)
    {
        if (i % 2 == 0)
        {
            suma = suma + i;
        }
    }

    printf ("\nLa suma de los pares entre 1 y %d es:%d \n",n,suma);

    return 0;
}
```

En el caso de las series existe un método sencillo para trasladar las mismas a una secuencia de instrucciones. En las series suma ($\sum$) la variable acumulador se inicializa a cero con objeto de ir sumando los términos de la serie utilizando el operador suma (+). Cada término se acumula asignando (=), en cada iteración, la suma del término y la variable acumulador.

Los valores inicial y final de la serie son respectivamente el valor inicial y la condición de finalización del bucle *for* (si la serie tiene como valor final infinito (∞) es necesario definir un valor final finito con el que obtener una aproximación de la serie).

En el caso de las series producto ($\prod$) el mecanismo es similar pero el acumulador se inicializa a 1 y el operador será el producto (*).

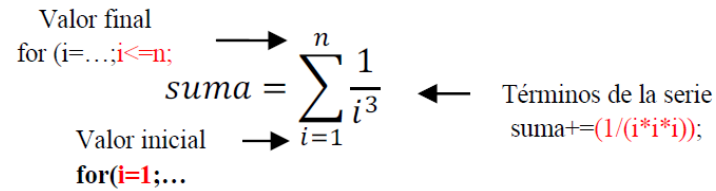


Fig. 1. Ejemplo de un sumatorio y valores en los que fijarse.

```

/* Ejemplo: Resolver la serie de la imagen anterior a partir
de un valor de n introducido por el usuario */

#include <stdio.h>

int main(void)
{
    int n, i;
    float suma=0.0;

    printf ("Introduzca numero de terminos de la serie:");
    scanf ("%d",&n);

    for (i=1;i<=n;i++)
    {
        // Cuidado con la división entre números enteros
        suma = suma + (1.0/(i*i*i));
    }

    printf ("La suma de la serie es: %f\n",suma);

    return 0;
}

```

El siguiente ejemplo resuelve un productorio que permite calcular el factorial de un número:

$$factorial = \prod_{i=1}^n i$$

```
/* Ejemplo: Resolver la serie de productos
a partir de un valor de n introducido por el usuario.
Observe que esta serie producto representa el factorial de n */

#include <stdio.h>

int main(void)
{
    int n, i, factorial=1;
    printf ("Introduzca un número entero:");
    scanf ("%d",&n);
    for (i=1;i<=n;i++)
    {
        factorial = factorial * i;
    }

    printf ("El factorial de %d es: %d\n",n,factorial);

    return 0;
}
```

4. Banderas (Flags)

Las banderas (flags) permiten comprobar/marcar si una propiedad lógica (verdadera o falsa) que debe comprobarse repetitivamente en muchos casos, se cumple o no. Por ejemplo, determinar si un número es primo.

En este tipo de problemas, se debe definir:

- Qué propiedad se va a verificar (verdadero o falso). En el ejemplo, si el número es primo, o si no lo es.
- Y cuál es la condición bajo la que se cumple. En el ejemplo, si encontramos un divisor del número (distinto de 1 y del propio número, sabemos que el número no es primo).

En el caso que tratamos, determinar que un número NO es primo es más fácil, porque es suficiente con hallar UN divisor distinto de la unidad y del propio número, para poder afirmar que el número no es primo. El caso más evidente es el de los número pares, verificar que NO son primos solo requiere comprobar que son divisibles por 2. Sin embargo, verificar que SI son primos, requiere comprobar todos los divisores.

En el ejemplo, vamos a suponer que el número que se comprueba es primo. La bandera toma el valor verdadero (1). Durante el bucle, se recorren posibles divisores y, si se encuentra uno, se cambia el valor de la bandera falso.

Al finalizar el bucle, se comprobará si la bandera tiene valor verdadero (es primo), o falso (NO es primo).

```
/* Ejemplo: Determinar si un número entero introducido por teclado es primo o no */

#include <stdio.h>

int main(void)
{
    int n,i;
    int es_primo=1;

    printf ("Introduzca un numero entero:");
    scanf ("%d",&n);

    for (i=2;i<n;i++)
    {
        if (n % i == 0)
        {
            es_primo=0;
        }
    }

    if (es_primo == 1)
    {
        printf ("El numero %d es primo\n",n);
    }
    else
    {
        printf ("El numero %d no es primo\n",n);
    }

    return 0;
}
```
